

CUSTOMER ONBOARDING AI

Task 6 Checkpoint

Beginner-Friendly • Enterprise-Grade Engineering Guide

FastAPI Domain Runtime and API-Backed UI

Checkpoint: a55c522

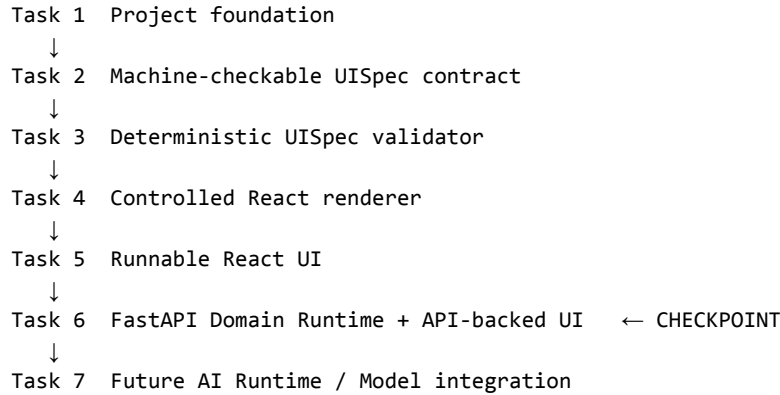
Commit: Task 6: Implement FastAPI domain runtime and API-backed UI

1. What This Checkpoint Means

This document records the formal checkpoint for Task 6. The implementation was already completed; this checkpoint verifies the reviewed state, commits it to Git, pushes it to the shared repository, and confirms that the repository is clean and synchronized.

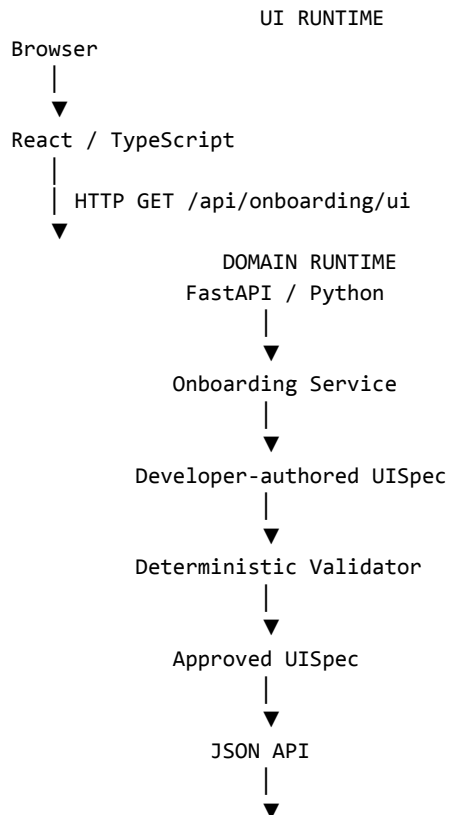
The checkpoint itself introduced no new functionality. Its purpose is reproducibility, traceability, and confidence that Task 6 can be treated as a stable engineering milestone.

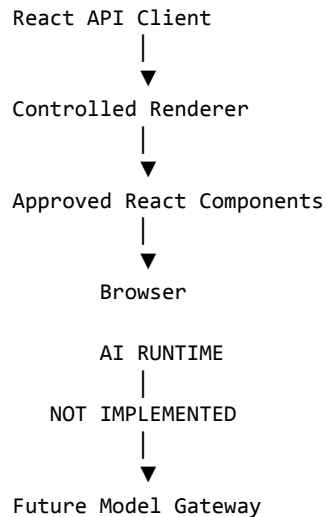
2. Task 6 in the Overall Incremental Journey



Task 6 is therefore an important architectural seam, but it is not yet the AI Runtime. FastAPI belongs to the Domain Runtime.

3. The Architecture at the Task 6 Checkpoint





The important separation is: React is the UI Runtime, FastAPI is the Domain Runtime, and the future model gateway/model is the AI Runtime.

4. What Was Verified

Checkpoint Item	Result	Evidence
Commit	PASS	a55c522
Commit message	PASS	Task 6: Implement FastAPI domain runtime and API-backed UI
Push	PASS	bc96e12..a55c522 main → main
Git status	PASS	Clean; synchronized with origin/main
Python tests	PASS	34/34
React tests	PASS	52/52
JSON Schema regression	PASS	12/12
Production build	PASS	tsc + Vite build
FastAPI /health	PASS	HTTP 200
FastAPI /api/onboarding/ui	PASS	HTTP 200; valid UISpec
Live validator check	PASS	validator.valid = True
Vite API proxy	PASS	HTTP 200 through localhost:5173
Security regression	PASS	Task 4 controls intact
AI/model integration	NOT PRESENT	No AI SDK/model gateway
Task 6A	NOT STARTED	No ContextPayload query seam
Task 7	NOT STARTED	No AI integration

5. Beginner Explanation: Why a Checkpoint Matters

In software engineering, finishing code is not the same as establishing a reliable milestone. A checkpoint creates a known-good state that another engineer can inspect, reproduce, compare, and build on.

- The code has been tested.
- The build has been tested.
- The API has been exercised.
- The browser integration has been exercised.

- Security controls have been rechecked.
- The exact code state has been committed.
- The commit has been pushed to the shared repository.
- The working tree is clean.

This gives the next task a stable starting point instead of building on an ambiguous or partially working state.

6. API Smoke Tests

```
GET http://127.0.0.1:8000/health
```

```
→ 200
```

```
→ {"status":"ok"}
```

```
GET http://127.0.0.1:8000/api/onboarding/ui
```

```
→ 200
```

```
→ UISpec body containing Hero + CTA + Text
```

```
Live response validation:
```

```
→ validator.valid = True
```

```
→ response body == developer-authored fixture = True
```

The second check is particularly useful: it verifies that the actual response being served by the running API is accepted by the same deterministic validator used by the domain service.

7. Browser / Vite Smoke Test

```
GET http://127.0.0.1:5173/
```

```
→ 200
```

```
GET http://127.0.0.1:5173/api/onboarding/ui
```

```
→ 200
```

```
→ Vite development proxy forwards the request to FastAPI
```

```
After verification:
```

```
→ FastAPI stopped
```

```
→ Vite stopped
```

```
→ ports 5173 and 8000 confirmed free
```

8. Security and Control Verification

The checkpoint confirmed that the controls introduced in earlier tasks remain intact.

- No dangerouslySetInnerHTML.
- No eval().
- No new Function().
- No dynamic import().
- App.tsx routes ready data through render(state.spec).
- The React application does not import componentRegistry to bypass the renderer.
- The React application does not call createElement directly.
- Rendered ready-state HTML contains the controlled CTA data-action.
- No inline on* event handlers or javascript: URLs.
- The running service does not import an AI/model SDK.
- No arbitrary component execution was introduced.

9. Dependency Review

The intended Task 6 Python dependencies are:

```
jsonschema>=4.20
pytest>=7.0
fastapi>=0.111
uvicorn[standard]>=0.30
httpx>=0.27
```

The React dependency set was unchanged from Task 5. No AI SDK, agent framework, ORM, authentication library, or cloud AI SDK was added.

- No anthropic
- No openai
- No langchain
- No cohere
- No mistralai
- No google.generativeai
- No boto3
- No llama_index
- No ollama
- No ORM
- No authentication library

10. What Was NOT Started

The checkpoint deliberately confirms that the following remain outside Task 6:

- No ContextPayload query parameters / Task 6A.
- No model gateway.
- No AiClient interface.
- No AI/model API calls.
- No authentication.
- No database.
- No document processing.
- No risk scoring.
- No human approval workflow.
- No production infrastructure.

This is an important enterprise discipline: do not quietly expand the scope of a checkpoint while trying to improve the architecture.

11. Git Checkpoint

```
Commit:
a55c522 Task 6: Implement FastAPI domain runtime and API-backed UI
```

```
Push:
bc96e12..a55c522  main -> main
```

Final state:

main synchronized with origin/main
working tree clean

The expected history now begins:

a55c522 Task 6: Implement FastAPI domain runtime and API-backed UI
bc96e12 Task 5: Implement minimal runnable React UI
1cb43a0 Task 4: Implement controlled UISpec renderer
31d4674 Task 3: Implement UISpec validator and tests
3824b07 Task 2: Formalise UISpec model contract
e289754 Task 1: Establish project foundation
2961344 Initial commit

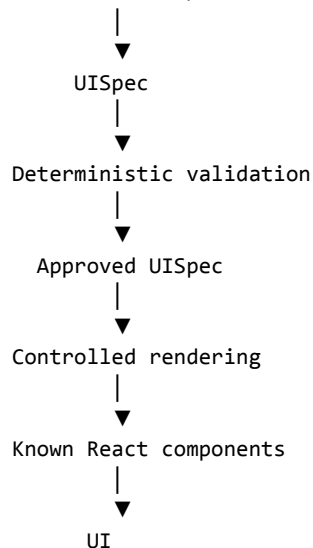
12. Enterprise Engineering Lessons

Task 6 demonstrates several enterprise-grade principles in a deliberately small implementation:

- Stable boundaries: UI, Domain, and future AI Runtime are separated.
- Validation before trust: the service does not return an unvalidated UISpec.
- Single rendering path: API data still reaches the browser through the controlled renderer.
- Fail visibly: the React client does not silently revert to the old hard-coded fixture on API failure.
- Minimal API surface: only the required health and onboarding UI endpoints are present.
- Dependency discipline: no framework or SDK was added merely because it might be useful later.
- Incremental delivery: the AI runtime remains a future task rather than being mixed into the domain seam.
- Checkpoint discipline: implementation and verification are separated from the commit/push milestone.

13. The Most Important Mental Model

Probabilistic / future AI world



The future AI model will not receive permission to directly create executable application behavior. It will eventually propose a structured representation that must pass the established controls.

14. Recommended Next Step

The checkpoint report identifies three possible next increments. They should remain separate tasks, rather than being combined:

1. Documentation backfill: add Task 4, Task 5, and Task 6 sections to docs/walkthrough.md.
2. ContextPayload request seam (Task 6A): accept segment, product_interest, and region while still returning a developer-authored fixture.
3. AiClient interface + FixtureAiClient: name the seam that the future AI gateway can occupy without changing the current wire behavior.

The key sequencing principle is to continue advancing in small, reviewable slices.

15. Exact Task 6 Checkpoint Prompt Used

The following is reproduced exactly as the checkpoint instruction used for Claude Code. It is intentionally preserved as the authoritative checkpoint prompt.

Proceed with the Task 6 checkpoint only.

IMPORTANT:

Task 6 implementation has already been completed and reviewed.

Do NOT implement any new functionality.

Do NOT start Task 6A.

Do NOT introduce ContextPayload request parameters.

Do NOT introduce a model gateway.

Do NOT introduce AI integration.

Do NOT modify the architecture.

Do NOT modify the model contract.

Do NOT modify the JSON Schema.

Do NOT modify the deterministic validator.

Do NOT modify the controlled renderer.

This task is ONLY:

VERIFY → CHECKPOINT → COMMIT → PUSH

```
=====
CURRENT CHECKPOINT
=====
```

The previous official checkpoint is:

bc96e12 Task 5: Implement minimal runnable React UI

Task 6 implementation currently exists in the working tree.

The purpose of this task is to establish the official Git checkpoint for Task 6.

```
=====
BEFORE CHECKPOINTING
=====
```

Inspect the current repository.

Confirm:

1. The repository is on the expected branch.
2. The previous checkpoint exists.
3. Task 6 implementation changes are present.
4. No Task 6A or Task 7 changes have been introduced.

Review the current diff.

Do not modify implementation unless there is an accidental artifact that must be removed.

```
=====
COLD-STATE VERIFICATION
=====
```

Run the complete verification from a clean working state.

PYTHON

Run:

```
python -m pytest -v
```

Expected:

34/34 tests passing

This includes:

- 24 Task 3 validator tests
- 10 Task 6 API tests

REACT

From ui/ run:

```
npm test
```

Expected:

52/52 tests passing

This includes:

Task 4 renderer/registry/security/determinism tests
Task 5 fixture tests
Task 6 API-client tests
Task 6 App tests

SCHEMA

Run the existing Task 2 JSON Schema verification.

Expected:

12/12 checks passing

BUILD

Run:

```
npm run build
```

Expected:

- TypeScript compilation succeeds
- Vite production build succeeds

API SMOKE TEST

Start the FastAPI server.

Verify:

GET /health

Expected:

HTTP 200

```
{
  "status": "ok"
}
```

Verify:

GET /api/onboarding/ui

Expected:

HTTP 200

The response must be the current developer-authored UISpec.

Feed the live API response back through the existing Python validator.

Confirm:

```
validator.valid == True
```

Confirm the response is equivalent to the developer-authored fixture.

BROWSER / VITE SMOKE TEST

Start the React development server.

Verify:

GET /

Expected:

HTTP 200

Verify:

GET /api/onboarding/ui

Expected:

HTTP 200 through the Vite development proxy.

Confirm that the browser application can obtain the UISpec from FastAPI.

Stop both development servers after verification.

Confirm ports are released.

=====

SECURITY REGRESSION

=====

Confirm that Task 4 security controls remain intact.

Verify that ui/src contains no:

- dangerouslySetInnerHTML
- eval()
- new Function()
- dynamic import()

Confirm:

- App.tsx still routes successful API data through render(...)
- App.tsx does not import componentRegistry
- App.tsx does not call createElement
- API data cannot cause arbitrary component execution
- rendered CTA does not contain inline JavaScript handlers
- no javascript: URLs are generated
- no AI SDK is imported by the service

=====

DEPENDENCY REVIEW

=====

Review:

requirements.txt

ui/package.json

Confirm that only the intended Task 6 dependencies were added.

Expected Python additions:

fastapi
uvicorn
httpx

Confirm no AI/model SDK has been added.

Confirm no:

- anthropic
- openai
- langchain
- cohere
- mistralai
- google.generativeai
- boto3
- llama_index
- ollama
- ORM
- authentication framework

has been introduced.

=====

DOCUMENTATION REVIEW

=====

Confirm that the Task 6 documentation updates are present.

Review:

README.md

docs/architecture.md

tests/README.md

ui/README.md

Do not update:

docs/walkthrough.md

The walkthrough will be updated separately.

```
=====
GIT HYGIENE
=====
```

Run:

```
git diff --check
```

Confirm there are no whitespace errors.

Review the full diff carefully.

Confirm there are no:

- node_modules
- dist
- __pycache__
- temporary files
- local environment files
- generated artifacts

that should be committed.

```
=====
CHECKPOINT COMMIT
=====
```

If and only if all verification passes, create exactly this commit:

Task 6: Implement FastAPI domain runtime and API-backed UI

Use:

```
git add ...
```

```
git commit -m "Task 6: Implement FastAPI domain runtime and API-backed UI"
```

Do not modify the commit message.

```
=====
PUSH
=====
```

Push the checkpoint to:

origin/main

Use:

git push origin main

Confirm that the push succeeds.

=====

FINAL VERIFICATION

=====

After pushing:

Run:

git status

Confirm:

- working tree clean
- branch synchronized with origin/main

Run:

git log --oneline -5

Confirm that the Task 6 commit is the latest commit.

Expected history should begin with:

<Task 6 commit> Task 6: Implement FastAPI domain runtime and API-backed UI
bc96e12 Task 5: Implement minimal runnable React UI
1cb43a0 Task 4: Implement controlled UISpec renderer
...

=====

IMPORTANT SCOPE CONFIRMATION

=====

Before finishing, explicitly confirm:

- No new functionality was implemented during this checkpoint task.
- Task 6A was NOT started.
- No ContextPayload request parameters were added.
- No model gateway was added.
- No AI integration was added.
- No authentication was added.
- No database was added.
- No document processing was added.
- No risk functionality was added.
- No approval workflow was added.
- No production infrastructure was added.

=====

FINAL REPORT

=====

Report:

1. Task 6 commit hash
2. Commit message
3. Push result
4. Final git status
5. Python test result
6. React test result
7. JSON Schema verification
8. Production build result
9. API smoke-test result
10. Browser / Vite smoke-test result
11. Security regression result
12. Dependency review
13. Confirmation that no new functionality was implemented in this checkpoint task
14. Confirmation that Task 6A was NOT started
15. Confirmation that Task 7 was NOT started
16. Final recommended next step

STOP.

Do not start another task.

Wait for human review.

16. Final Checkpoint Status

TASK 6 — CHECKPOINTED ✓

Commit a55c522 is pushed to origin/main. Verification passed, the working tree is clean, and Task 7 has not started.